



Titel

- Buchtausch-App Datenbank
Phase 2 – Erarbeitungs-/Reflexionsphase

Mohamad Almoussa
Matrikelnummer: IU14121494
Kurs: DLBDSPBDM01_D



Projektbeschreibung

- Ziel dieses Projekts ist die Entwicklung einer relationalen Datenbank für eine Buchtausch-App.
- Benutzer:innen können Bücher anbieten, ausleihen und verwalten.
- Die Datenbank speichert Informationen über Bücher, Benutzer:innen, Ausleihen, Bewertungen, Nachrichten und Standorte.
- Die Implementierung wurde mit MariaDB und SQL umgesetzt.

+

•

○

Verwendete Technologien

- MariaDB
- SQL
- XAMPP
- Visual Studio Code
- dbdiagram.io

Entity-Relationship-Modell (ERM)

- Das ER-Modell zeigt die wichtigsten Entitäten und Beziehungen der Buchtausch-App.
- Die Datenbank enthält Tabellen für Benutzer:innen, Bücher, Ausleihen, Bewertungen, Nachrichten, Kategorien und Standorte.

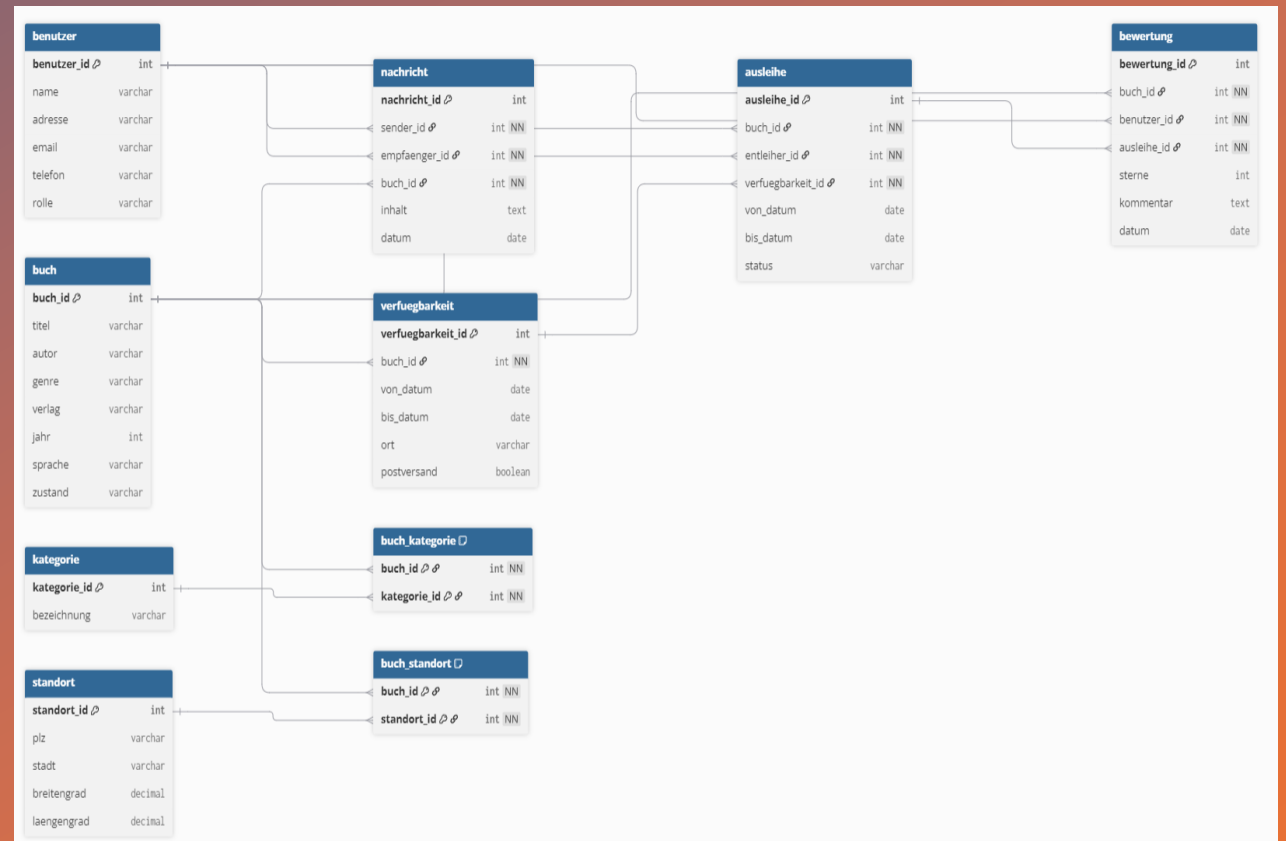


Tabelle Benutzer

- Diese Tabelle speichert Informationen über die Benutzer:innen der Buchtausch-App, einschließlich Name, Adresse, E-Mail, Telefonnummer und Rolle.
- Durch den CHECK-Constraint wird sichergestellt, dass nur die Rollen „Anbieter“, „Entleiher“ oder „Admin“ gespeichert werden können.

```
CREATE TABLE benutzer (  
  benutzer_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  adresse VARCHAR(255) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  telefon VARCHAR(30),  
  rolle VARCHAR(50) NOT NULL,  
  CHECK (rolle IN ('Anbieter', 'Entleiher', 'Admin'))  
);
```

```
INSERT INTO benutzer (name, adresse, email, telefon, rolle) VALUES  
(  
  'Anna Müller', 'Hauptstraße 10, Berlin', 'anna.mueller@example.de', '01711111111', 'Anbieter'),  
(  
  'Max Schmidt', 'Gartenweg 5, Hamburg', 'max.schmidt@example.de', '01722222222', 'Entleiher'),  
(  
  'Sofia Weber', 'Bahnhofstraße 3, Köln', 'sofia.weber@example.de', '01733333333', 'Anbieter'),  
(  
  'Lukas Fischer', 'Ringstraße 8, München', 'lukas.fischer@example.de', '01744444444', 'Entleiher'),
```

```
MariaDB [buchtausch_app]> SELECT * FROM benutzer;  
+-----+-----+-----+-----+-----+-----+  
| benutzer_id | name      | adresse                | email                | telefon  | rolle  |  
+-----+-----+-----+-----+-----+-----+  
| 1 | Anna Müller | Hauptstraße 10, Berlin | anna.mueller@example.de | 01711111111 | Anbieter |  
| 2 | Max Schmidt | Gartenweg 5, Hamburg  | max.schmidt@example.de  | 01722222222 | Entleiher |  
| 3 | Sofia Weber | Bahnhofstraße 3, Köln | sofia.weber@example.de  | 01733333333 | Anbieter |  
| 4 | Lukas Fischer | Ringstraße 8, München | lukas.fischer@example.de | 01744444444 | Entleiher |  
| 5 | Emma Wagner | Schulstraße 12, Essen | emma.wagner@example.de  | 01755555555 | Anbieter |  
| 6 | Noah Becker | Marktplatz 2, Dortmund | noah.becker@example.de  | 01766666666 | Entleiher |  
| 7 | Mia Hoffmann | Wiesenweg 7, Bochum  | mia.hoffmann@example.de | 01777777777 | Anbieter |  
| 8 | Leon Koch   | Kirchstraße 4, Duisburg | leon.koch@example.de    | 01788888888 | Entleiher |  
| 9 | Clara Richter | Parkallee 9, Düsseldorf | clara.richter@example.de | 01799999999 | Admin |  
| 10 | Paul Neumann | Bergstraße 6, Gelsenkirchen | paul.neumann@example.de | 01611111111 | Anbieter |  
+-----+-----+-----+-----+-----+-----+  
10 rows in set (0.000 sec)
```

Tabelle Buch

- Diese Tabelle speichert Informationen über die Bücher der Buchtausch-App, einschließlich Titel, Autor:in, Genre, Verlag, Veröffentlichungsjahr, Sprache und Zustand des Buches.
- Zusätzliche CHECK-Constraints stellen sicher, dass nur gültige Werte für den Zustand gespeichert werden und das Veröffentlichungsjahr größer als 0 ist.

```
CREATE TABLE buch (  
    buch_id INT AUTO_INCREMENT PRIMARY KEY,  
    titel VARCHAR(150) NOT NULL,  
    autor VARCHAR(100) NOT NULL,  
    genre VARCHAR(80),  
    verlag VARCHAR(100),  
    jahr INT,  
    sprache VARCHAR(50) NOT NULL,  
    zustand VARCHAR(50) NOT NULL,  
    CHECK (jahr IS NULL OR jahr > 0),  
    CHECK (zustand IN ('sehr gut', 'gut', 'gebraucht'))  
);
```

```
INSERT INTO buch (titel, autor, genre, verlag, jahr, sprache, zustand) VALUES  
(  
'Der kleine Prinz', 'Antoine de Saint-Exupéry', 'Roman', 'Karl Rauch Verlag', 1943, 'Deutsch', 'gut'),  
(  
'Harry Potter und der Stein der Weisen', 'J. K. Rowling', 'Fantasy', 'Carlsen', 1998, 'Deutsch', 'sehr gut'),  
(  
'1984', 'George Orwell', 'Dystopie', 'Ullstein', 1949, 'Deutsch', 'gut'),
```

```
MariaDB [buchtausch_app]> SELECT * FROM buch;
```

buch_id	titel	autor	genre	verlag	jahr	sprache	zustand
1	Der kleine Prinz	Antoine de Saint-Exupéry	Roman	Karl Rauch Verlag	1943	Deutsch	gut
2	Harry Potter und der Stein der Weisen	J. K. Rowling	Fantasy	Carlsen	1998	Deutsch	sehr gut
3	1984	George Orwell	Dystopie	Ullstein	1949	Deutsch	gut
4	Die Verwandlung	Franz Kafka	Erzählung	Reclam	1915	Deutsch	gebraucht
5	Clean Code	Robert C. Martin	Informatik	MITP	2008	Englisch	sehr gut
6	Datenbanken verstehen	Hans Müller	Fachbuch	Springer	2020	Deutsch	gut
7	Der Alchimist	Paulo Coelho	Roman	Diogenes	1988	Deutsch	gut
8	Sofies Welt	Jostein Gaarder	Philosophie	Hanser	1991	Deutsch	gebraucht
9	Java ist auch eine Insel	Christian Ullenboom	Informatik	Rheinwerk	2022	Deutsch	sehr gut
10	Effektives Lernen	Peter Brown	Sachbuch	Beltz	2014	Deutsch	gut

```
10 rows in set (0.001 sec)
```

Tabelle Verfügbarkeit

- Diese Tabelle speichert Informationen über die Verfügbarkeit der Bücher, einschließlich Ausleihzeitraum, Abholort und der Möglichkeit des Postversands.
- Durch den Foreign Key wird sichergestellt, dass jede Verfügbarkeit einem existierenden Buch zugeordnet ist. Zusätzlich prüft ein CHECK-Constraint, dass das Enddatum nicht vor dem Startdatum liegt.

```
CREATE TABLE verfuegbarkeit (  
    verfuegbarkeit_id INT AUTO_INCREMENT PRIMARY KEY,  
    buch_id INT NOT NULL,  
    von_datum DATE NOT NULL,  
    bis_datum DATE NOT NULL,  
    ort VARCHAR(100) NOT NULL,  
    postversand BOOLEAN NOT NULL DEFAULT FALSE,  
    FOREIGN KEY (buch_id) REFERENCES buch(buch_id)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT,  
    CHECK (bis_datum >= von_datum)  
);
```

```
INSERT INTO verfuegbarkeit (buch_id, von_datum, bis_datum, ort, postversand) VALUES  
(1, '2026-05-01', '2026-06-01', 'Berlin', TRUE),  
(2, '2026-05-03', '2026-06-03', 'Hamburg', FALSE),  
(3, '2026-05-05', '2026-06-05', 'Köln', TRUE),
```

```
MariaDB [buchtausch_app]> SELECT * FROM verfuegbarkeit;
```

verfuegbarkeit_id	buch_id	von_datum	bis_datum	ort	postversand
1	1	2026-05-01	2026-06-01	Berlin	1
2	2	2026-05-03	2026-06-03	Hamburg	0
3	3	2026-05-05	2026-06-05	Köln	1
4	4	2026-05-07	2026-06-07	München	0
5	5	2026-05-09	2026-06-09	Essen	1
6	6	2026-05-11	2026-06-11	Dortmund	0
7	7	2026-05-13	2026-06-13	Bochum	1
8	8	2026-05-15	2026-06-15	Duisburg	0
9	9	2026-05-17	2026-06-17	Düsseldorf	1
10	10	2026-05-19	2026-06-19	Gelsenkirchen	0

```
10 rows in set (0.000 sec)
```


Tabelle Ausleihe

- Diese Tabelle speichert Informationen über Ausleihvorgänge der Bücher, einschließlich Entleiher:in, Ausleihzeitraum und aktuellem Status der Ausleihe.
- Die Tabelle verwendet mehrere Foreign Keys, um Beziehungen zu Büchern, Benutzer:innen und Verfügbarkeiten herzustellen. Zusätzlich sorgen CHECK-Constraints dafür, dass nur gültige Statuswerte gespeichert werden und das Rückgabedatum nicht vor dem Ausleihdatum liegt.

```
CREATE TABLE ausleihe (  
  ausleihe_id INT AUTO_INCREMENT PRIMARY KEY,  
  buch_id INT NOT NULL,  
  entleiher_id INT NOT NULL,  
  verfuegbarkeit_id INT NOT NULL,  
  von_datum DATE NOT NULL,  
  bis_datum DATE NOT NULL,  
  status VARCHAR(50) NOT NULL,  
  FOREIGN KEY (buch_id) REFERENCES buch(buch_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  FOREIGN KEY (entleiher_id) REFERENCES benutzer(benutzer_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  FOREIGN KEY (verfuegbarkeit_id) REFERENCES verfuegbarkeit(verfuegbarkeit_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  CHECK (status IN ('aktiv', 'zurückgegeben', 'storniert')),  
  CHECK (bis_datum >= von_datum)  
);
```

```
INSERT INTO ausleihe (buch_id, entleiher_id, verfuegbarkeit_id, von_datum, bis_datum, status) VALUES  
(1, 2, 1, '2026-05-02', '2026-05-20', 'aktiv'),  
(2, 4, 2, '2026-05-04', '2026-05-22', 'aktiv'),  
(3, 6, 3, '2026-05-06', '2026-05-24', 'zurückgegeben'),
```

```
MariaDB [buchtausch_app]> SELECT * FROM ausleihe;
```

ausleihe_id	buch_id	entleiher_id	verfuegbarkeit_id	von_datum	bis_datum	status
1	1	2	1	2026-05-02	2026-05-20	aktiv
2	2	4	2	2026-05-04	2026-05-22	aktiv
3	3	6	3	2026-05-06	2026-05-24	zurückgegeben
4	4	8	4	2026-05-08	2026-05-26	aktiv
5	5	2	5	2026-05-10	2026-05-28	zurückgegeben
6	6	4	6	2026-05-12	2026-05-30	aktiv
7	7	6	7	2026-05-14	2026-06-01	aktiv
8	8	8	8	2026-05-16	2026-06-03	zurückgegeben
9	9	2	9	2026-05-18	2026-06-05	aktiv
10	10	4	10	2026-05-20	2026-06-07	aktiv

```
10 rows in set (0.001 sec)
```


Tabelle Bewertung

- Diese Tabelle speichert Bewertungen und Kommentare zu ausgeliehenen Büchern, einschließlich Sternebewertung, Kommentar und Bewertungsdatum.
- Durch die Foreign Keys wird sichergestellt, dass jede Bewertung mit einem existierenden Buch, einer Benutzerin bzw. einem Benutzer und einem Ausleihvorgang verknüpft ist. Der CHECK-Constraint stellt außerdem sicher, dass nur Bewertungen zwischen 1 und 5 Sternen gespeichert werden können.

```
CREATE TABLE bewertung (  
  bewertung_id INT AUTO_INCREMENT PRIMARY KEY,  
  buch_id INT NOT NULL,  
  benutzer_id INT NOT NULL,  
  ausleihe_id INT NOT NULL UNIQUE,  
  sterne INT NOT NULL,  
  kommentar TEXT,  
  datum DATE NOT NULL,  
  FOREIGN KEY (buch_id) REFERENCES buch(buch_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  FOREIGN KEY (benutzer_id) REFERENCES benutzer(benutzer_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  FOREIGN KEY (ausleihe_id) REFERENCES ausleihe(ausleihe_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  CHECK (sterne BETWEEN 1 AND 5)  
);
```

```
INSERT INTO bewertung (buch_id, benutzer_id, ausleihe_id, sterne, kommentar, datum) VALUES  
(3, 6, 3, 5, 'Sehr gutes Buch und schnelle Übergabe.', '2026-05-25'),  
(5, 2, 5, 4, 'Sehr hilfreich für Programmierung.', '2026-05-29'),  
(8, 8, 8, 4, 'Interessantes Buch, Zustand war okay.', '2026-06-04'),
```

```
MariaDB [buchtausch_app]> SELECT * FROM bewertung;
```

bewertung_id	buch_id	benutzer_id	ausleihe_id	sterne	kommentar	datum
1	3	6	3	5	Sehr gutes Buch und schnelle Übergabe.	2026-05-25
2	5	2	5	4	Sehr hilfreich für Programmierung.	2026-05-29
3	8	8	8	4	Interessantes Buch, Zustand war okay.	2026-06-04
4	1	2	1	5	Schöne Geschichte und gutes Exemplar.	2026-05-21
5	2	4	2	5	Sehr spannend und sauber.	2026-05-23
6	4	8	4	3	Buch war lesbar, aber etwas alt.	2026-05-27
7	6	4	6	4	Gutes Fachbuch für Datenbanken.	2026-05-31
8	7	6	7	5	Sehr inspirierend.	2026-06-02
9	9	2	9	5	Perfekt für Java-Lernen.	2026-06-06
10	10	4	10	4	Guter Inhalt zum Lernen.	2026-06-08

```
10 rows in set (0.002 sec)
```

Tabelle Kategorie

- Diese Tabelle speichert die verschiedenen Kategorien der Bücher, wie zum Beispiel Roman, Fantasy oder Informatik.
- Durch den UNIQUE-Constraint wird sichergestellt, dass jede Kategorie nur einmal in der Datenbank vorhanden ist.

```
CREATE TABLE kategorie (  
    kategorie_id INT AUTO_INCREMENT PRIMARY KEY,  
    bezeichnung VARCHAR(100) NOT NULL UNIQUE  
);
```

```
INSERT INTO kategorie (bezeichnung) VALUES  
('Roman'),  
('Fantasy'),  
('Dystopie'),
```

```
MariaDB [buchtausch_app]> SELECT * FROM kategorie;
```

kategorie_id	bezeichnung
1	Roman
2	Fantasy
3	Dystopie
4	Erzählung
5	Informatik
6	Fachbuch
7	Philosophie
8	Sachbuch
9	Klassiker
10	Lernen

10 rows in set (0.001 sec)

Tabelle Buch_Kategorie

- Diese Tabelle stellt die Many-to-Many-Beziehung zwischen Büchern und Kategorien dar.
- Ein Buch kann mehreren Kategorien zugeordnet werden und eine Kategorie kann mehrere Bücher enthalten. Der zusammengesetzte Primärschlüssel verhindert doppelte Zuordnungen.

```
CREATE TABLE buch_kategorie (  
  buch_id INT NOT NULL,  
  kategorie_id INT NOT NULL,  
  PRIMARY KEY (buch_id, kategorie_id),  
  FOREIGN KEY (buch_id) REFERENCES buch(buch_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  FOREIGN KEY (kategorie_id) REFERENCES kategorie(kategorie_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT  
);
```

```
INSERT INTO buch_kategorie (buch_id, kategorie_id) VALUES  
(1, 1),  
(2, 2),  
(3, 3),
```

```
MariaDB [buchtausch_app]> SELECT  
->   b.titel,  
->   k.bezeichnung AS kategorie  
-> FROM buch b  
-> JOIN buch_kategorie bk ON b.buch_id = bk.buch_id  
-> JOIN kategorie k ON bk.kategorie_id = k.kategorie_id;
```

titel	kategorie
Der kleine Prinz	Roman
Harry Potter und der Stein der Weisen	Fantasy
1984	Dystopie
Die Verwandlung	Erzählung
Clean Code	Informatik
Datenbanken verstehen	Fachbuch
Der Alchimist	Roman
Sofies Welt	Philosophie
Java ist auch eine Insel	Informatik
Effektives Lernen	Lernen

```
10 rows in set (0.001 sec)
```

Tabelle Nachricht

- Diese Tabelle speichert Nachrichten zwischen Benutzer:innen der Buchtausch-App bezüglich bestimmter Bücher.
- Die Tabelle enthält Beziehungen zu Sender:in, Empfänger:in und Buch. Der CHECK-Constraint stellt sicher, dass eine Person keine Nachricht an sich selbst senden kann.

```
CREATE TABLE Nachricht (  
  Nachricht_id INT AUTO_INCREMENT PRIMARY KEY,  
  sender_id INT NOT NULL,  
  empfaenger_id INT NOT NULL,  
  buch_id INT NOT NULL,  
  inhalt TEXT NOT NULL,  
  datum DATE NOT NULL,  
  FOREIGN KEY (sender_id) REFERENCES benutzer(benutzer_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  FOREIGN KEY (empfaenger_id) REFERENCES benutzer(benutzer_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  FOREIGN KEY (buch_id) REFERENCES buch(buch_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  CHECK (sender_id <> empfaenger_id)  
);
```

```
INSERT INTO Nachricht (sender_id, empfaenger_id, buch_id, inhalt, datum) VALUES  
(2, 1, 1, 'Hallo, ist das Buch noch verfügbar?', '2026-05-01'),  
(4, 3, 2, 'Kann ich das Buch morgen abholen?', '2026-05-03'),  
(6, 5, 3, 'Ist Postversand möglich?', '2026-05-05'),
```

```
MariaDB [buchtausch_app]> SELECT  
->   n.Nachricht_id,  
->   sender.name AS sender,  
->   empfaenger.name AS empfaenger,  
->   b.titel AS buch,  
->   n.inhalt,  
->   n.datum  
-> FROM Nachricht n  
-> JOIN benutzer sender ON n.sender_id = sender.benutzer_id  
-> JOIN benutzer empfaenger ON n.empfaenger_id = empfaenger.benutzer_id  
-> JOIN buch b ON n.buch_id = b.buch_id;
```

nachricht_id	sender	empfaenger	buch	inhalt	datum
1	Max Schmidt	Anna Müller	Der kleine Prinz	Hallo, ist das Buch noch verfügbar?	2026-05-01
2	Lukas Fischer	Sofia Weber	Harry Potter und der Stein der Weisen	Kann ich das Buch morgen abholen?	2026-05-03
3	Noah Becker	Emma Wagner	1984	Ist Postversand möglich?	2026-05-05
4	Leon Koch	Mia Hoffmann	Die Verwandlung	Wie ist der Zustand des Buches?	2026-05-07
5	Max Schmidt	Emma Wagner	Clean Code	Ich interessiere mich für Clean Code.	2026-05-09
6	Lukas Fischer	Anna Müller	Datenbanken verstehen	Wann wäre eine Übergabe möglich?	2026-05-11
7	Noah Becker	Sofia Weber	Der Alchimist	Ist das Buch auf Deutsch?	2026-05-13
8	Leon Koch	Emma Wagner	Sofies Welt	Kann ich es für zwei Wochen ausleihen?	2026-05-15
9	Max Schmidt	Mia Hoffmann	Java ist auch eine Insel	Ich möchte Java lernen, ist das Buch geeignet?	2026-05-17
10	Lukas Fischer	Paul Neumann	Effektives Lernen	Ist das Buch noch in Gelsenkirchen verfügbar?	2026-05-19

10 rows in set (0.002 sec)

Tabelle Standort

- Diese Tabelle speichert Standortinformationen für die geografische Suche nach Büchern.
- Gespeichert werden Postleitzahl, Stadt sowie Breitengrad und Längengrad. Die CHECK-Constraints stellen sicher, dass nur gültige geografische Koordinaten gespeichert werden können.

```
CREATE TABLE standort (  
    standort_id INT AUTO_INCREMENT PRIMARY KEY,  
    plz VARCHAR(10) NOT NULL,  
    stadt VARCHAR(100) NOT NULL,  
    breitengrad DECIMAL(10,7) NOT NULL,  
    laengengrad DECIMAL(10,7) NOT NULL,  
    CHECK (breitengrad BETWEEN -90 AND 90),  
    CHECK (laengengrad BETWEEN -180 AND 180)  
);
```

```
INSERT INTO standort (plz, stadt, breitengrad, laengengrad) VALUES  
(  
    '10115', 'Berlin', 52.5320000, 13.3849000),  
(  
    '20095', 'Hamburg', 53.5503000, 10.0006000),  
(  
    '50667', 'Köln', 50.9375000, 6.9603000),
```

```
MariaDB [buchtausch_app]> SELECT  
->     standort_id,  
->     plz,  
->     stadt,  
->     breitengrad,  
->     laengengrad  
-> FROM standort;
```

standort_id	plz	stadt	breitengrad	laengengrad
1	10115	Berlin	52.5320000	13.3849000
2	20095	Hamburg	53.5503000	10.0006000
3	50667	Köln	50.9375000	6.9603000
4	80331	München	48.1374000	11.5755000
5	45127	Essen	51.4556000	7.0116000
6	44135	Dortmund	51.5136000	7.4653000
7	44787	Bochum	51.4818000	7.2162000
8	47051	Duisburg	51.4344000	6.7623000
9	40213	Düsseldorf	51.2254000	6.7763000
10	45879	Gelsenkirchen	51.5177000	7.0857000

10 rows in set (0.001 sec)

Tabelle Buch_Standort

- Diese Tabelle stellt die Beziehung zwischen Büchern und Standorten dar.
- Dadurch kann ein Buch einem bestimmten Standort zugeordnet werden, um eine geografische Suche innerhalb der Buchtausch-App zu ermöglichen. Der zusammengesetzte Primärschlüssel verhindert doppelte Zuordnungen.

```
CREATE TABLE buch_standort (  
  buch_id INT NOT NULL,  
  standort_id INT NOT NULL,  
  PRIMARY KEY (buch_id, standort_id),  
  FOREIGN KEY (buch_id) REFERENCES buch(buch_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT,  
  FOREIGN KEY (standort_id) REFERENCES standort(standort_id)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT  
);
```

```
INSERT INTO buch_standort (buch_id, standort_id) VALUES  
(1, 1),  
(2, 2),  
(3, 3),
```

```
MariaDB [buchtausch_app]> SELECT  
->   b.titel,  
->   s.stadt,  
->   s.plz  
-> FROM buch b  
-> JOIN buch_standort bs ON b.buch_id = bs.buch_id  
-> JOIN standort s ON bs.standort_id = s.standort_id;
```

titel	stadt	plz
Der kleine Prinz	Berlin	10115
Harry Potter und der Stein der Weisen	Hamburg	20095
1984	Köln	50667
Die Verwandlung	München	80331
Clean Code	Essen	45127
Datenbanken verstehen	Dortmund	44135
Der Alchimist	Bochum	44787
Sofies Welt	Duisburg	47051
Java ist auch eine Insel	Düsseldorf	40213
Effektives Lernen	Gelsenkirchen	45879

```
10 rows in set (0.001 sec)
```


Testfall 1 – Bücher mit Verfügbarkeit

- Diese Abfrage zeigt alle Bücher zusammen mit ihren Verfügbarkeitsinformationen, einschließlich Zeitraum, Ort und Postversand.

```
MariaDB [buchtausch_app]> SELECT  
->   b.buch_id,  
->   b.titel,  
->   b.autor,  
->   v.von_datum,  
->   v.bis_datum,  
->   v.ort,  
->   v.postversand  
-> FROM buch b  
-> JOIN verfuegbarkeit v ON b.buch_id = v.buch_id;
```

buch_id	titel	autor	von_datum	bis_datum	ort	postversand
1	Der kleine Prinz	Antoine de Saint-Exupéry	2026-05-01	2026-06-01	Berlin	1
2	Harry Potter und der Stein der Weisen	J. K. Rowling	2026-05-03	2026-06-03	Hamburg	0
3	1984	George Orwell	2026-05-05	2026-06-05	Köln	1
4	Die Verwandlung	Franz Kafka	2026-05-07	2026-06-07	München	0
5	Clean Code	Robert C. Martin	2026-05-09	2026-06-09	Essen	1
6	Datenbanken verstehen	Hans Müller	2026-05-11	2026-06-11	Dortmund	0
7	Der Alchimist	Paulo Coelho	2026-05-13	2026-06-13	Bochum	1
8	Sofies Welt	Jostein Gaarder	2026-05-15	2026-06-15	Duisburg	0
9	Java ist auch eine Insel	Christian Ullenboom	2026-05-17	2026-06-17	Düsseldorf	1
10	Effektives Lernen	Peter Brown	2026-05-19	2026-06-19	Gelsenkirchen	0

10 rows in set (0.001 sec)

Testfall 2 – Ausleihen mit Benutzer und Buch

- Diese Abfrage zeigt alle Ausleihvorgänge zusammen mit den entsprechenden Benutzer:innen und Büchern.

```
MariaDB [buchtausch_app]> SELECT
->   a.ausleihe_id,
->   bu.name AS entleiher,
->   b.titel AS buch,
->   a.von_datum,
->   a.bis_datum,
->   a.status
-> FROM ausleihe a
-> JOIN benutzer bu ON a.entleiher_id = bu.benutzer_id
-> JOIN buch b ON a.buch_id = b.buch_id;
```

ausleihe_id	entleiher	buch	von_datum	bis_datum	status
1	Max Schmidt	Der kleine Prinz	2026-05-02	2026-05-20	aktiv
2	Lukas Fischer	Harry Potter und der Stein der Weisen	2026-05-04	2026-05-22	aktiv
3	Noah Becker	1984	2026-05-06	2026-05-24	zurückgegeben
4	Leon Koch	Die Verwandlung	2026-05-08	2026-05-26	aktiv
5	Max Schmidt	Clean Code	2026-05-10	2026-05-28	zurückgegeben
6	Lukas Fischer	Datenbanken verstehen	2026-05-12	2026-05-30	aktiv
7	Noah Becker	Der Alchimist	2026-05-14	2026-06-01	aktiv
8	Leon Koch	Sofies Welt	2026-05-16	2026-06-03	zurückgegeben
9	Max Schmidt	Java ist auch eine Insel	2026-05-18	2026-06-05	aktiv
10	Lukas Fischer	Effektives Lernen	2026-05-20	2026-06-07	aktiv

10 rows in set (0.001 sec)

Testfall 3 – Bewertungen mit Benutzer und Buch

- Diese Abfrage zeigt Bewertungen und Kommentare zu Büchern zusammen mit den entsprechenden Benutzer:innen.

```
MariaDB [buchtausch_app]> SELECT
->   bw.bewertung_id,
->   bu.name AS benutzer,
->   b.titel AS buch,
->   bw.sterne,
->   bw.kommentar,
->   bw.datum
-> FROM bewertung bw
-> JOIN benutzer bu ON bw.benutzer_id = bu.benutzer_id
-> JOIN buch b ON bw.buch_id = b.buch_id;
```

bewertung_id	benutzer	buch	sterne	kommentar	datum
1	Noah Becker	1984	5	Sehr gutes Buch und schnelle Übergabe.	2026-05-25
2	Max Schmidt	Clean Code	4	Sehr hilfreich für Programmierung.	2026-05-29
3	Leon Koch	Sofies Welt	4	Interessantes Buch, Zustand war okay.	2026-06-04
4	Max Schmidt	Der kleine Prinz	5	Schöne Geschichte und gutes Exemplar.	2026-05-21
5	Lukas Fischer	Harry Potter und der Stein der Weisen	5	Sehr spannend und sauber.	2026-05-23
6	Leon Koch	Die Verwandlung	3	Buch war lesbar, aber etwas alt.	2026-05-27
7	Lukas Fischer	Datenbanken verstehen	4	Gutes Fachbuch für Datenbanken.	2026-05-31
8	Noah Becker	Der Alchimist	5	Sehr inspirierend.	2026-06-02
9	Max Schmidt	Java ist auch eine Insel	5	Perfekt für Java-Lernen.	2026-06-06
10	Lukas Fischer	Effektives Lernen	4	Guter Inhalt zum Lernen.	2026-06-08

10 rows in set (0.000 sec)

Testfall 4 – Bücher nach Standort

- Diese Abfrage zeigt Bücher zusammen mit den zugehörigen Standortinformationen für die geografische Suche.

```
MariaDB [buchtausch_app]> SELECT
->   b.titel,
->   s.stadt,
->   s.plz,
->   s.breitengrad,
->   s.laengengrad
-> FROM buch b
-> JOIN buch_standort bs ON b.buch_id = bs.buch_id
-> JOIN standort s ON bs.standort_id = s.standort_id;
```

titel	stadt	plz	Breitengrad	Laengengrad
Der kleine Prinz	Berlin	10115	52.5320000	13.3849000
Harry Potter und der Stein der Weisen	Hamburg	20095	53.5503000	10.0006000
1984	Köln	50667	50.9375000	6.9603000
Die Verwandlung	München	80331	48.1374000	11.5755000
Clean Code	Essen	45127	51.4556000	7.0116000
Datenbanken verstehen	Dortmund	44135	51.5136000	7.4653000
Der Alchimist	Bochum	44787	51.4818000	7.2162000
Sofies Welt	Duisburg	47051	51.4344000	6.7623000
Java ist auch eine Insel	Düsseldorf	40213	51.2254000	6.7763000
Effektives Lernen	Gelsenkirchen	45879	51.5177000	7.0857000

```
10 rows in set (0.000 sec)
```

Testfall 11 – Durchschnittliche Bewertung pro Buch

- Diese Abfrage berechnet die durchschnittliche Bewertung pro Buch und zeigt zusätzlich die Anzahl der Bewertungen an.

```
MariaDB [buchtausch_app]> SELECT
->   b.titel,
->   ROUND(AVG(bw.sterne), 2) AS durchschnittliche_bewertung,
->   COUNT(bw.bewertung_id) AS anzahl_bewertungen
-> FROM buch b
-> LEFT JOIN bewertung bw ON b.buch_id = bw.buch_id
-> GROUP BY b.buch_id, b.titel
-> ORDER BY durchschnittliche_bewertung DESC;
```

titel	durchschnittliche_bewertung	anzahl_bewertungen
Java ist auch eine Insel	5.00	1
Der kleine Prinz	5.00	1
1984	5.00	1
Der Alchimist	5.00	1
Harry Potter und der Stein der Weisen	5.00	1
Clean Code	4.00	1
Effektives Lernen	4.00	1
Datenbanken verstehen	4.00	1
Sofies Welt	4.00	1
Die Verwandlung	3.00	1

10 rows in set (0.001 sec)

Testfall 12 – Mehrfachausleihen pro Buch

- Für diesen Testfall wurde ein zusätzlicher Ausleihvorgang eingefügt, um Mehrfachausleihen realistisch prüfen zu können.

```
MariaDB [buchtausch_app]> INSERT INTO ausleihe
-> (buch_id, entleiher_id, verfuegbarkeit_id, von_datum, bis_datum, status)
-> VALUES
-> (1, 4, 1, '2026-06-02', '2026-06-15', 'aktiv');
Query OK, 1 row affected (0.004 sec)
```

```
MariaDB [buchtausch_app]> SELECT
->     b.buch_id,
->     b.titel,
->     COUNT(a.ausleihe_id) AS anzahl_ausleihen
-> FROM buch b
-> JOIN ausleihe a ON b.buch_id = a.buch_id
-> GROUP BY b.buch_id, b.titel
-> HAVING COUNT(a.ausleihe_id) > 1
-> ORDER BY anzahl_ausleihen DESC;
```

```
+-----+-----+-----+
| buch_id | titel          | anzahl_ausleihen |
+-----+-----+-----+
|      1 | Der kleine Prinz |                2 |
+-----+-----+-----+
1 row in set (0.000 sec)
```

+

-

```
MariaDB [buchtausch_app]> SHOW INDEX FROM buch;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
buch	0	PRIMARY	1	buch_id	A	10	NULL	NULL		BTREE		

1 row in set (0.001 sec)

Reflexion und Verbesserungen

- Im Rahmen dieses Projekts wurde eine relationale Datenbank für eine Buchtausch-App erfolgreich entwickelt und implementiert.
- Die Datenbank enthält mehrere miteinander verbundene Tabellen für Benutzer:innen, Bücher, Ausleihen, Bewertungen, Nachrichten, Kategorien und Standorte. Durch Foreign Keys, CHECK-Constraints und Indizes wurde die Datenintegrität sowie die Effizienz der Datenbank verbessert.
- Zusätzlich wurden verschiedene Testfälle durchgeführt, um reale Nutzungsszenarien zu simulieren und die Funktionalität der Datenbank zu überprüfen.
- In Zukunft könnte die Datenbank um weitere Funktionen erweitert werden, zum Beispiel:
 - Reservierungssysteme
 - automatische Benachrichtigungen
 - erweiterte Standortsuche
 - Benutzerrechte und Sicherheit
 - Integration in eine Web- oder Mobile-Anwendung